

21. if et else : deux chemins

Le programme ne fait pas tout : il choisit ce qu'il fait, image après image.

1. Ce qu'il faut comprendre

`if (condition) { ... }` exécute le bloc **seulement si** la condition est vraie. `else { ... }` donne le chemin **sinon**. Un seul des deux blocs s'exécute, jamais les deux.

`bouton(A)` rend 1 quand A est enfoncé, 0 sinon. Pour `if`, **0 veut dire faux, et tout le reste veut dire vrai** — il n'y a pas d'autre définition de « vrai » dans la machine.

Les huit touches de la Game Boy, toutes lues par `bouton()` : A et B (les deux boutons ronds), HAUT, BAS, GAUCHE et DROITE (la croix), START et SELECT (les deux petits boutons du milieu). Chacune a sa ligne à l'écran, avec son nom.

Tant qu'on tient une touche, son nom clignote, et il reste affiché dès qu'on la lâche. Pour clignoter, il faut une horloge : `images()` compte les images depuis l'allumage, soixante par seconde. `images() / 16` avance d'un toutes les seize images, et `% 2` n'en garde que le reste : **0, 1, 0, 1...** — c'est le rythme du clignotement.

C'est un `if` dans un `if` : le premier demande si la touche est tenue ; le second, seulement dans ce cas, choisit entre des espaces (autant que de lettres : quatre pour « HAUT », six pour « GAUCHE ») et le nom. 1 veut dire vrai, 0 faux : on n'a rien à comparer. Le `else` du premier réécrit le nom — sans lui, une touche lâchée au mauvais moment laisserait son nom effacé.

Les huit `if` sont **indépendants** : on peut tenir plusieurs touches à la fois — la croix et A, par exemple, comme pour sauter en courant — et plusieurs noms clignent en même temps.

Le test est refait **à chaque image**, soixante fois par seconde, parce qu'il est dans la boucle du jeu. C'est ce qui rend le programme réactif : il repose sans cesse la même question, et la réponse change quand le joueur agit.

Au clavier de l'ordinateur : les **flèches** pour la croix, **X** pour A, **Z** pour B (ou l'inverse, dans les réglages), **Entrée** pour START et **Maj** pour SELECT — et chacune se change dans ⚙ Options, « Les touches du clavier ».

Chaque `texte()` attend son moment : la console n'écrit à l'écran que pendant un court instant à chaque image — le chapitre 12 l'explique. Huit touches, huit écritures : un tour de boucle dure donc **huit images**, et l'on voit un tout petit retard entre l'appui et le premier clignotement. Le chapitre 12 montre comment n'écrire que ce qui change.

À toi : fais clignoter la touche A deux fois plus vite. Que faut-il changer dans `images() / 16` ?

Les fonctions de cette leçon — et le tuto de chacune

- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <bouton>` → [0.65.5. La fonction bouton\(\) — lire la manette](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1115 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l’a demandé — et ce qu’il a laissé, parce que rien ne l’appelle :

Gravé dans la cartouche :

démarrage	49 o	← la console, à l’allumage – la mise en route de la console
main()	522 o	← démarrage – ta fonction
VBlank	8 o	← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage	28 o	← image() – attend l’image suivante
AttendreVBlank	16 o	← démarrage – attend que l’écran soit libre
AttendreAcces	12 o	← EcrireTexte – attend de pouvoir écrire une case
EcrireTexte	10 o	← texte() – écrit un texte, lettre par lettre
CopierTuiles	67 o	← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage – vide les deux cartes de l’écran
LireManette	37 o	← bouton() – lit les huit boutons
Tuiles	240 o	← CopierTuiles – 15 lettres sur 43
"TIENS UNE TOUCHE"	16 o	← texte() – un texte gravé
" "	1 o	← texte() – un texte gravé
"A"	1 o	← texte() – un texte gravé
"A"	1 o	← texte() – un texte gravé
" "	1 o	← texte() – un texte gravé
"B"	1 o	← texte() – un texte gravé
"B"	1 o	← texte() – un texte gravé
" "	4 o	← texte() – un texte gravé
"HAUT"	4 o	← texte() – un texte gravé
"HAUT"	4 o	← texte() – un texte gravé
" "	3 o	← texte() – un texte gravé
"BAS"	3 o	← texte() – un texte gravé
"BAS"	3 o	← texte() – un texte gravé
" "	6 o	← texte() – un texte gravé
"GAUCHE"	6 o	← texte() – un texte gravé
"GAUCHE"	6 o	← texte() – un texte gravé
" "	6 o	← texte() – un texte gravé
"DROITE"	6 o	← texte() – un texte gravé
"DROITE"	6 o	← texte() – un texte gravé
" "	5 o	← texte() – un texte gravé
"START"	5 o	← texte() – un texte gravé
"START"	5 o	← texte() – un texte gravé
" "	6 o	← texte() – un texte gravé
"SELECT"	6 o	← texte() – un texte gravé
"SELECT"	6 o	← texte() – un texte gravé

Lettres gravées : ABCDEGHILNORSTU

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, DonneesTransfert, Notes

Fonctions du langage appelées :

texte()	× 25	275 o sur place, dans main()	+ EcrireTexte (10 o)
bouton()	× 8	40 o sur place, dans main()	+ LireManette (37 o)
images()	× 8	16 o sur place, dans main()	
image()	× 1	3 o sur place, dans main()	+ AttendreImage (28 o)

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// CE PROGRAMME : écrit le nom des 8 touches de la console.
// Quand tu TIENS une touche, son nom clignote. Quand tu la lâches, il reste affiché.
//
// Ce qui est nouveau : if et else.
// if (condition) ... = « SI la condition est vraie, fais ceci »
// else ...          = « SINON, fais cela »
// Un seul des deux chemins est pris, jamais les deux.

#include <texte>    // écrit un texte à l'écran
#include <bouton>  // lit un bouton de la manette
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {                                // Le jeu commence ici.
    texte(1, 2, "TIENS UNE TOUCHE");        // La consigne, écrite une fois.

    while (true) {                          // Répète pour toujours :
        image();                            // attend l'image suivante (60 par seconde).

        // Tenue : son nom clignote. Lâchée : il reste affiché.

        // ---- La touche A (colonne 4, ligne 5). Expliquée en détail ;
        //      les 7 autres touches plus bas font exactement la même chose.
        if (bouton(A)) {                    // SI la touche A est enfoncée en ce moment...
                                            // bouton(A) vaut 1 (vrai) quand elle est tenue,
                                            // 0 (faux) sinon.
            if ((images() / 16) % 2) {      // ... alors on choisit selon l'heure :
                                            // images() compte les images depuis le début.
                                            // / 16 : ce nombre change toutes les 16 images.
                                            // % 2 : le reste de la division par 2, 0 ou 1.
                                            // Exemple : image 40 : 40 / 16 = 2, 2 % 2 = 0.
                                            //           image 50 : 50 / 16 = 3, 3 % 2 = 1.
                                            // Donc : 16 images à 1, 16 images à 0, etc.
                                            // à 1 : on efface la lettre (un espace).
                texte(4, 5, " ");          // à 0 :
            } else {                        // on la réécrit. Effacer, écrire... : ça clignote.
                texte(4, 5, "A");
            }
        } else {                            // SINON (A n'est pas enfoncée) :
            texte(4, 5, "A");              // la lettre reste simplement écrite.
        }

        // ---- La touche B (ligne 6) : même chose.
        if (bouton(B)) {
            if ((images() / 16) % 2) {
                texte(4, 6, " ");          // 1 espace pour effacer 1 lettre
            } else {
                texte(4, 6, "B");
            }
        } else {
            texte(4, 6, "B");
        }

        // ---- HAUT (ligne 7).
        if (bouton(HAUT)) {
            if ((images() / 16) % 2) {
                texte(4, 7, "    ");      // 4 espaces pour effacer les 4 lettres de HAUT
            } else {
                texte(4, 7, "HAUT");
            }
        } else {
            texte(4, 7, "HAUT");
        }

        // ---- BAS (ligne 8).
        if (bouton(BAS)) {
            if ((images() / 16) % 2) {
                texte(4, 8, "   ");       // 3 espaces pour BAS
            } else {
                texte(4, 8, "BAS");
            }
        } else {
            texte(4, 8, "BAS");
        }

        // ---- GAUCHE (ligne 9).
        if (bouton(GAUCHE)) {
            if ((images() / 16) % 2) {
                texte(4, 9, "      ");    // 6 espaces pour GAUCHE
            } else {
                texte(4, 9, "GAUCHE");
            }
        }
    }
}
```

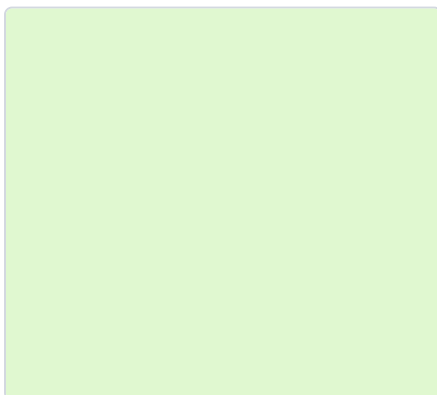
```

    }
  } else {
    texte(4, 9, "GAUCHE");
  }
  // ---- DROITE (ligne 10).
  if (bouton(DROITE)) {
    if ((images() / 16) % 2) {
      texte(4, 10, "      ");          // 6 espaces pour DROITE
    } else {
      texte(4, 10, "DROITE");
    }
  } else {
    texte(4, 10, "DROITE");
  }
  // ---- START (ligne 11).
  if (bouton(START)) {
    if ((images() / 16) % 2) {
      texte(4, 11, "      ");          // 5 espaces pour START
    } else {
      texte(4, 11, "START");
    }
  } else {
    texte(4, 11, "START");
  }
  // ---- SELECT (ligne 12).
  if (bouton(SELECT)) {
    if ((images() / 16) % 2) {
      texte(4, 12, "      ");          // 6 espaces pour SELECT
    } else {
      texte(4, 12, "SELECT");
    }
  } else {
    texte(4, 12, "SELECT");
  }
}                                     // fin de la boucle : on repart à image()
}                                     // fin de main

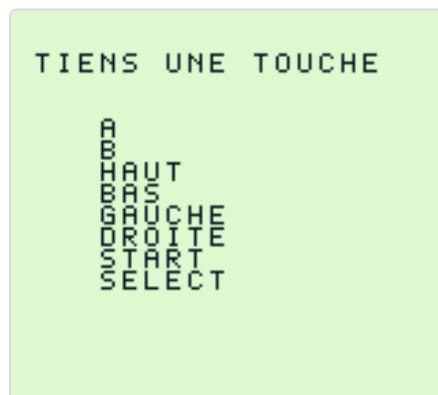
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



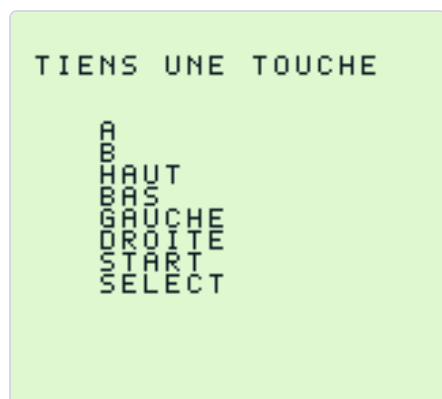
la toute première image — la console vient de démarrer — image 1



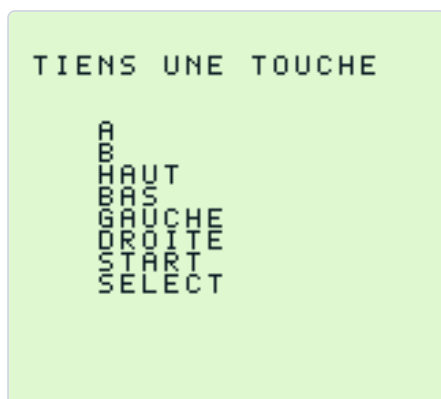
un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

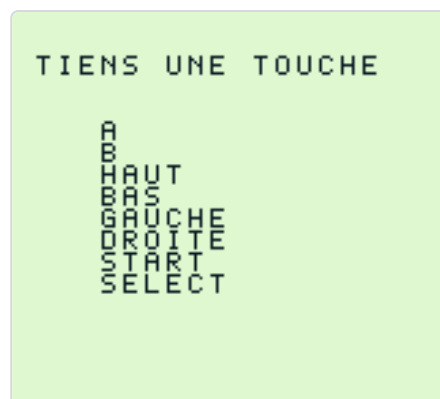
Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



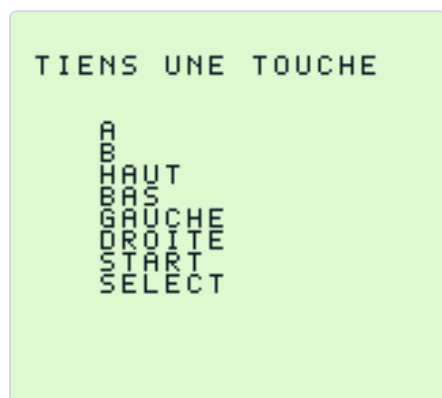
A — l'écran n'a pas bougé



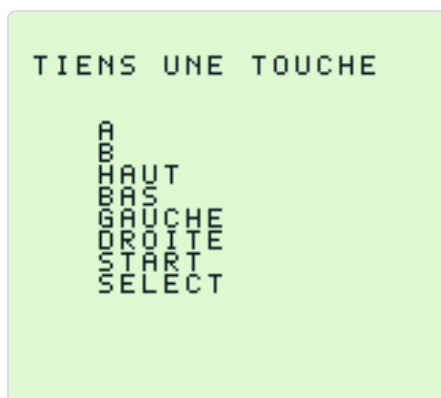
B — l'écran n'a pas bougé



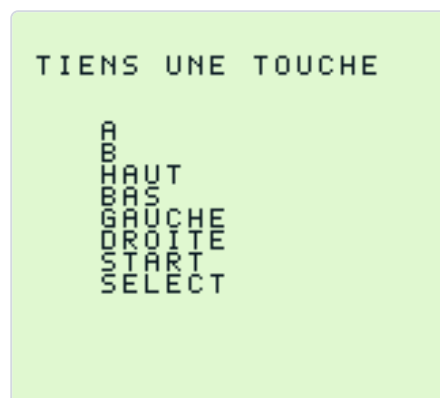
DROITE — l'écran n'a pas bougé



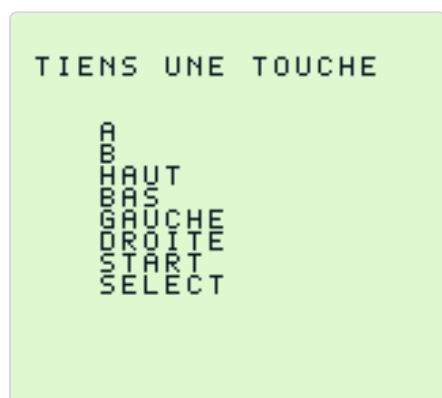
GAUCHE — l'écran n'a pas bougé



HAUT — l'écran n'a pas bougé



BAS — l'écran n'a pas bougé



START — l'écran n'a pas bougé

5. Ce qu'on doit voir

Les huit noms à l'écran ; une touche tenue fait clignoter le sien, relâchée il reste affiché.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ au repos, les huit noms sont à l'écran

✓ A tenu : son nom clignote — tantôt là, tantôt vide — et lui seul

- ✓ A lâché : son nom reste affiché
- ✓ B tenu : son nom clignote — tantôt là, tantôt vide — et lui seul
- ✓ B lâché : son nom reste affiché
- ✓ HAUT tenu : son nom clignote — tantôt là, tantôt vide — et lui seul
- ✓ HAUT lâché : son nom reste affiché
- ✓ BAS tenu : son nom clignote — tantôt là, tantôt vide — et lui seul
- ✓ BAS lâché : son nom reste affiché
- ✓ GAUCHE tenu : son nom clignote — tantôt là, tantôt vide — et lui seul
- ✓ GAUCHE lâché : son nom reste affiché
- ✓ DROITE tenu : son nom clignote — tantôt là, tantôt vide — et lui seul
- ✓ DROITE lâché : son nom reste affiché
- ✓ START tenu : son nom clignote — tantôt là, tantôt vide — et lui seul
- ✓ START lâché : son nom reste affiché
- ✓ SELECT tenu : son nom clignote — tantôt là, tantôt vide — et lui seul
- ✓ SELECT lâché : son nom reste affiché
- ✓ DROITE et A ensemble : les deux clignent

Ensuite : **22. Une comparaison est un nombre** — chapitre 4 : Les conditions : choisir. · gameboy3 — écrire une cartouche Game Boy en C++